

Report
v. 1.02

Customer
Liquid Bitcoin Foundation



Smart Contract Audit

Token Distributor

13 September 2025



Report prepared by
ABDK
Consulting

Contents

1	Changelog	3
2	Introduction	4
3	Project scope	5
4	Methodology	6
5	Our findings	7
6	Moderate Issues	8
	CVF-1. FIXED	8
	CVF-3. FIXED	8
	CVF-5. FIXED	9
7	Minor Issues	10
	CVF-2. INFO	10
	CVF-4. INFO	10
	CVF-6. INFO	11
	CVF-10. INFO	11
	CVF-11. INFO	11
	CVF-12. INFO	12
	CVF-13. INFO	12
	CVF-14. FIXED	13
	CVF-15. INFO	13
	CVF-16. INFO	13
	CVF-17. INFO	14
	CVF-18. INFO	14
	CVF-19. FIXED	14
	CVF-20. INFO	15

1 Changelog

#	Date	Author	Description
0.1	12.09.25	D. Khovratovich	Initial Draft
1.02	13.09.25	D. Khovratovich	Final Version

2 Introduction

All modifications to this document are prohibited. Violators will be prosecuted to the full extent of the U.S. law.

The following document provides the result of the audit performed by ABDK Consulting (Mikhail Vladimirov and Dmitry Khovratovich) at the customer request. The audit goal is a general review of the smart contracts structure, critical/major bugs detection and issuing the general recommendations.

3 Project scope

We were asked to review the new Token Distributor functionality of Liquid Bitcoin Foundation, which is based on the earlier Agora Distributor contract.

We were asked to review:

- [Original Token Distributor Code.](#)
- [Token Distributor Code with Fixes;](#)

Files:

TokenDistributor.sol

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment.** The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis.** Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis.** For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis.** The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** contain code style, best practices and other suggestions.

5 Our findings

We found 0 critical, 0 major, and a few less important issues.



Fixed 5 out of 17 issues

6 Moderate Issues

CVF-1 FIXED

- **Category** Unclear behavior
- **Source** TokenDistributor.sol

Description This effectively forbids the account to get tokens twice if there are two leaves.

Recommendation Document why it is not possible or change the logic.

Client Comment *Restrict user to be able to do only one claim is intentional.*

```
207 hasClaimed[_account] = true;
```

CVF-3 FIXED

- **Category** Readability
- **Source** TokenDistributor.sol

Description This parameter is not documented

```
222 bytes calldata _proof
```

```
277 bytes calldata _proof
```


CVF-5 FIXED

- **Category** Suboptimal

- **Source** TokenDistributor.sol

Description Hashing a leaf twice seems to be redundant.

Recommendation Consider hashing once, probably with some salt to avoid collisions with non-leaf tree nodes. Use 'encodePacked' to reduce the hash cost

```
326 // Generate the leaf
    bytes32 leaf = keccak256(
        bytes.concat(
            keccak256(abi.encode(_account, _amount, ACCOUNT_TYPE_ADDRESS
            ↪ ))
```

```
351 // Generate the leaf
    bytes32 leaf = keccak256(
        bytes.concat(
            keccak256(abi.encode(_account, _amount, ACCOUNT_TYPE_BYTES32
            ↪ ))
    )
```

7 Minor Issues

CVF-2 INFO

- **Category** Readability

- **Source** TokenDistributor.sol

Description It is unclear that the "hasClaimed" check is performed inside the "_validateClaim" call.

Recommendation Refactor the code to check and update the "hasClaimed" flag at the same place.

Client Comment *This is purely a readability matter*

```
204 _validateClaim(_account, _amount, _merkleProof);
```

```
207 hasClaimed[_account] = true;
```

CVF-4 INFO

- **Category** Readability

- **Source** TokenDistributor.sol

Description It is unclear that the "hasClaimedByProof" check is performed inside the "_validateClaimByProof" call.

Recommendation Refactor the code to check and update the "hasClaimedByProof" flag at the same place.

Client Comment *This is purely a readability matter*

```
224 _validateClaimWithProof(
```

```
233 hasClaimedByProof[_account] = true;
```

CVF-6 INFO

- **Category** Suboptimal

- **Source** TokenDistributor.sol

Description The code of this function largely duplicates that of '_validateClaim()'

Recommendation Consider extracting common parts to a single function

Client Comment *This two functions have the same flow but parameter data types are slightly different so I do not see any merits in extracting the common part of it.*

```
338 function _validateClaimWithProof(
```

CVF-10 INFO

- **Category** Procedural

- **Source** TokenDistributor.sol

Description Consider specifying as "^0.8.0" unless there is something special regarding this particular version.

Client Comment *We prefer to set compiler version explicitly to make results predictable and compatible with all EVM chain we are considering as our targets*

```
2 pragma solidity 0.8.28;
```

CVF-11 INFO

- **Category** Bad naming

- **Source** TokenDistributor.sol

Description Events are usually named via nouns, such as "Claim" or "Withdrawal".

Client Comment *We prefer to keep event names the way they are now*

```
23 event Claimed(address indexed user, uint256 amount);
```

```
29 event ClaimedWithProof(
```

```
38 event Withdrawn(address indexed owner, uint256 amount);
```

```
43 event VaultChanged(address indexed oldVault, address indexed  
    ↪ newVault);
```

CVF-12 INFO

- **Category** Suboptimal

- **Source** TokenDistributor.sol

Description The old value parameters are redundant, as their values could be derived from the previous events.

Client Comment *Both old and new parameters are added for convenience. This events are not supposed to happen frequently so it will be easier to use them instead of searching for previous similar event*

```
43 event VaultChanged(address indexed oldVault, address indexed  
    ↪ newVault);
```

```
48 event ApproverChanged(
```

```
56 event PauserChanged(address indexed oldPauser, address indexed  
    ↪ newPauser);
```

CVF-13 INFO

- **Category** Suboptimal

- **Source** TokenDistributor.sol

Description These errors could be made more useful by adding certain parameters into them.

Client Comment *We consider the current version of events as sufficient.*

```
62 error InvalidAmount();  
error AlreadyClaimed();  
error InvalidMerkleProof();  
error InvalidToken();  
error InvalidMerkleRoot();
```

```
68 error ClaimFinished();  
error ClaimNotFinished();  
70 error StakingNotEnabled();  
error WrongStakeAmount();  
error WrongClaimEnd();  
error Unauthorized();  
error WrongAddress();  
error ClaimWithProofNotEnabled();  
error InvalidProof();  
error OnlyRecipientCanStake();
```

CVF-14 FIXED

- **Category** Bad naming

- **Source** TokenDistributor.sol

Description UPPER_CASE identifiers are conventionally used for constants.

Recommendation Use camelCase identifiers

```
91 bytes32 public immutable MERKLE_ROOT;
```

```
94 IERC20 public immutable TOKEN;
```

```
97 uint256 public immutable CLAIM_END;
```

```
116 address public APPROVER;
```

CVF-15 INFO

- **Category** Bad datatype

- **Source** TokenDistributor.sol

Description The type for this argument should be "IERC20".

```
135 address _token,
```

CVF-16 INFO

- **Category** Bad datatype

- **Source** TokenDistributor.sol

Description The type for this argument should be "IERC4626".

```
138 address _vault,
```

CVF-17 INFO

- **Category** Suboptimal

- **Source** TokenDistributor.sol

Description These checks are redundant, as it is anyway possible to pass invalid values.

Recommendation Remove these checks.

Client Comment *I disagree. We added this check intentionally to decrease probability of mistake when deploying the contract. The merkle root and token address cannot be changed later so there will be no chance to fix them after the deployment.*

```
142 if (_token == address(0)) revert InvalidToken();  
    if (_merkleRoot == bytes32(0)) revert InvalidMerkleRoot();  
  
145 if (_pauser == address(0)) revert WrongAddress();
```

CVF-18 INFO

- **Category** Suboptimal

- **Source** TokenDistributor.sol

Description This check is redundant, as it is anyway possible to pass a dead pauser address.

Recommendation Remove this check.

Client Comment *We prefer to keep it the way it is now.*

```
173 if (newPauser == address(0)) {
```

CVF-19 FIXED

- **Category** Documentation

- **Source** TokenDistributor.sol

Description No signature involved in this function.

Recommendation Consider changing the comment

```
195 /// @notice Claim tokens using a signature and merkle proof.  
  
239 /// @notice Claim tokens using a signature and merkle proof and  
    ↪ stake them with predefined vault.  
  
251 /// @notice Claim tokens using a signature and merkle proof and  
    ↪ stake part of them with predefined vault.
```

CVF-20 INFO

- **Category** Bad datatype
- **Source** TokenDistributor.sol

Description The argument type should be "IERC4626".

```
300 function changeVault(address _newVault) external onlyOwner {
```



ABDK

Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 40 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting